

RAPPORT D'INTERVENTION · 21 AOÛT 2026

Déploiement Dughu Web

Préparation de la branche `production` pour une mise en ligne sur `testing.dughu.com` — six commits, trois blocages levés, un serveur écarté.

Dépôt `Dughu/DUGHU-WEB-3.0` Branche `production` HEAD `74eda2d` Poussée `oui`

Où on en est

APPLICATION	INFRASTRUCTURE	COMMITTS
Prête Le build passe sans avertissement. Serveur autonome vérifié en local.	En attente Le serveur pressenti a été écarté. Un VPS dédié reste à provisionner.	6 Tous poussés sur <code>origin/production</code>, arbre de travail propre.

La branche `production` part de `freddy` et n'a reçu que des changements de déploiement — aucune modification fonctionnelle de l'application, à une exception près : le renommage rendu obligatoire par Next 16, décrit plus bas.

Il ne manque qu'une machine. Tout le reste — configuration, script de déploiement, sonde de santé, documentation — est en place et versionné.

Comment on y est arrivé

Le travail a suivi une découverte progressive : chaque étape a invalidé une hypothèse de la précédente. C'est la raison d'être de ce rapport.

01 Analyse de `main`

Premier examen : Next.js 16, PostgreSQL via Prisma, 33 modèles, NextAuth, et des uploads écrits sur le disque local. Conclusion de l'époque : il faut un VPS avec base de données et volume persistant.

02 Bascule sur `freddy` — analyse à refaire

134 fichiers d'écart. Cette branche a supprimé toute la couche base de données : plus de Prisma, plus de PostgreSQL, plus de NextAuth. L'API Laravel Dughu devient la seule source de vérité, la session tient dans deux cookies, et plus aucune route n'écrit sur le disque.

L'application devient entièrement sans état. Ni base, ni volume, ni migration : le déploiement s'en trouve radicalement simplifié.

03 Création de `production`

Trois blocages de build levés, la configuration de déploiement écrite, 207 fichiers d'outillage retirés du suivi Git.

04 Cadrage du domaine

Cible fixée à `testing.dughu.com` adossé à l'API de recette `apitest.dughu.com`. Vérification DNS : `dughu.com` est déjà sur Cloudflare, et un wildcard `*.dughu.com` existe — sans enregistrement explicite, le sous-domaine pointait vers la mauvaise cible.

05 Inspection du serveur — et demi-tour

L'IP fournie s'est révélée être celle du serveur principal de Dughu. Sous Apache, saturé en mémoire. La procédure écrite jusque-là devenait dangereuse pour la production existante ; elle a été réécrite pour Apache, puis le serveur a été écarté.

Les trois blocages levés

La branche `freddy` contenait déjà un `Dockerfile` et un `docker-compose.yml`. Aucun des deux n'était exploitable en l'état.

CORRIGÉ `docker-compose.yml` invalide

Le fichier avait perdu toute son indentation — du YAML plat, que Docker refuse de lire.

```
$ docker compose config
services must be a mapping
```

Réécrit et validé. Ses commentaires évoquaient encore une base Neon et un service PostgreSQL optionnel — vestiges de `main`, supprimés.

CORRIGÉ `middleware.ts` cassait le build

La convention `middleware` est dépréciée en Next 16 au profit de `proxy`. Le simple renommage du fichier ne suffit pas : la fonction exportée doit l'être aussi, ce que le build a signalé sans ambiguïté.

```
$ npx next build
./src/proxy.ts
Proxy is missing expected function export name
This function is what Next.js runs for every request handled by this
proxy (previously called middleware).
```

C'est la seule modification de code applicatif de toute l'intervention. Le comportement de la garde d'authentification est identique.

CORRIGÉ Clé `eslint` inconnue

`next.config.ts` déclarait une clé `eslint` qui n'existe plus en Next 16 — avertissement `Unrecognized key(s) in object` à chaque build. Supprimée.

Ce qui est livré

FICHIERS AJOUTÉS OU RÉÉCRITS

FICHIER	RÔLE
src/app/api/health	Sonde /api/health , utilisée par Docker, Compose et le script de déploiement
.env.example	Les 9 variables réellement lues par le code, documentées
deploy/nginx/dughu.conf	Vhost de référence — TLS, streaming, en-têtes de sécurité
deploy/apache/...conf	Variante pour un serveur déjà servi par Apache
deploy/deploy.sh	Build avant bascule, attente de la sonde, arrêt propre en cas d'échec
Dockerfile	Node 22, utilisateur non-root, HEALTHCHECK intégré
docker-compose.yml	Publication sur 127.0.0.1 uniquement, rotation des logs
README.md	Réécrit — il contenait encore des marqueurs de conflit Git

Décisions de conception

— La sonde n'appelle pas l'API Dughu.

Elle
indique
que
le
processus
Next.js
répond,
pas
que
le
backend
distant
est
joignable

—
sinon
une
panne
côté
API
ferait
redémarrer
en
boucle
un
conteneur
pourtant
sain.

- Le conteneur n'écoute qu'en loopback.

Le ufw
rever
proxy
est
le
seul
point
d'ent
publi
C'est
aussi
ce
qui
neutr
le
contc
classi
d'
par
Docker.

- Le script construit avant d'arrêter l'ancienne image.

Si
le
build
échoue,
le
service
en
cours
continue
de
tourner.

— La compression reste assurée par Next.

Un
premier
réglage
la
déléguait
à
nginx
; le
proxy
n'étant
plus
garanti,
la
garder
côté
application
la
rend
correcte
dans
tous
les
cas.

Hygiène du dépôt

207 fichiers d'outillage d'agents (`.agents/` , `.claude/` , `.windsurf/`) ont été retirés du suivi. Ils avaient été ajoutés au `.gitignore` , ce qui restait sans effet : une règle d'ignore ne s'applique jamais à un fichier déjà indexé.

Un point laissé délibérément en l'état

Le dossier `public/uploads/` — 7 fichiers hérités — avait d'abord été supprimé, plus rien n'y écrivait sur cette branche. Décision annulée : le code résout encore les chemins `/uploads/...` vers ce dossier, et d'anciens contenus stockés côté API Dughu peuvent les référencer. Le gain était de 960 Ko, le risque était des images cassées en production.

Le serveur écarté

L'IP `203.161.53.131` avait été désignée comme cible. L'inspection par SSH a montré qu'il ne s'agissait pas d'une machine disponible, mais du serveur principal de Dughu : 17 vhosts actifs, dont `dughu.com`, `api.dughu.com`, `apitest.dughu.com`, `akwaplay.com`, `backoffice` et `kibana`.

```
$ free -h
              total        used        free      shared  buff/cache   available
Mem:           11Gi       10Gi       229Mi        195Mi         939Mi        676Mi
Swap:              0B           0B           0B

$ ps -eo rss,comm --sort=-rss | head -4
6811 MB  java      ← Elasticsearch (Kibana)
1005 MB  mysqld
546 MB   node
400 MB   apache2
```

Un `next build` demande environ 2 Go. Avec 676 Mo disponibles et aucun swap, le noyau aurait tué un processus pour récupérer de la mémoire — et les cibles les plus probables sont précisément `java` et `mysqld`. Autrement dit : une panne de production déclenchée par un déploiement de recette.

CONSTATS DE L'INSPECTION

POINT	CONSTAT	VERDICT
Mémoire	676 Mo dispo · 0 swap	BLOQUANT
Docker	Absent	À INSTALLER
Node	v12.22.9	NEXT 16 EXIGE ≥ 20.9
Clone existant	Sur <code>main</code> , pas <code>production</code>	OBSOLÈTE
Modules Apache	proxy, proxy_http, headers, rewrite, ssl	DÉJÀ ACTIFS
Port 3000	Libre	OK
Pare-feu	80/443 restreints aux 15 pages Cloudflare	BIEN CONFIGURÉ

Deux instructions retirées de la documentation

La procédure initiale contenait `sudo apt install -y nginx` et `sudo ufw enable`. Sur cette machine, la première aurait disputé le port 80 à Apache et mis les 17 sites hors ligne ; la seconde aurait posé un jeu de règles par-dessus un filtrage déjà en place, avec un risque de coupure des sites ou de l'accès SSH. Les deux ont été supprimées, avec l'explication en tête du README.

Rien n'a été modifié sur ce serveur : l'intervention s'est limitée à des commandes de lecture. Le clone obsolète dans `/var/www/DUGHU-WEB-3.0` est resté intact.

Ce qui est vérifié, et ce qui ne l'est pas

La distinction compte : une partie de la configuration n'a pas pu être exécutée depuis le poste de développement.

ÉTAT DES VÉRIFICATIONS

ÉLÉMENT	RÉSULTAT	
<code>next build</code>	37 pages, aucun avertissement	VÉRIFIÉ
Serveur autonome	<code>/api/health</code> → 200 · <code>/login</code> → 200 · <code>/home</code> → 307	VÉRIFIÉ
Compression	17 349 → ~4 500 octets	VÉRIFIÉ
En-tête <code>X-Powered-By</code>	Absent	VÉRIFIÉ
<code>docker compose config</code>	Valide, variables résolues	VÉRIFIÉ
Build de l'image Docker	Démon Docker indisponible sur le poste	NON VÉRIFIÉ
Syntaxe nginx	nginx absent — accolades équilibrées seulement	NON VÉRIFIÉ
Syntaxe Apache	<code>apache2ctl</code> absent — balises équilibrées seulement	NON VÉRIFIÉ

Les trois éléments non vérifiés disposent chacun d'un filet : `deploy.sh` s'interrompt en affichant les logs si l'image ne se construit pas, et `nginx -t` comme `apache2ctl configtest` refusent de recharger une configuration invalide. Ces commandes sont dans la procédure, avant tout rechargement.

La suite

Le serveur à commander

CARACTÉRISTIQUES DU VPS

RESSOURCE	CIBLE	POURQUOI
CPU	2 vCPU	Suffisant, le build est parallélisé mais court
RAM	4 Go	L'exécution tient dans 1 Go ; c'est le build, à ~2 Go, qui dimensionne
Disque	20 Go	Image et dépendances
OS	Ubuntu 22.04 / 24.04	Procédure écrite pour Debian et dérivés

Les 4 Go sont le point qui compte. Avec 2 Go, il faudra ajouter du swap avant le premier build.

Les étapes restantes

- 1 Provisionner le VPS et ouvrir un accès SSH.
- 2 Corriger l'enregistrement dans Cloudflare : il pointe encore sur l'ancienne cible. Le laisser en `testing` le temps d'obtenir le certificat.
DNS only

- 3 Cloner `production`, remplir le `.env` — il manque `DUGHU_API_KEY` et les identifiants SMTP — puis lancer `./deploy/deploy.sh`
--
no-pull

- 4 Poser le vhost nginx, vérifier avec `nginx -t`, recharger, puis `certbot --nginx -d testing.dughu.com`

5

Repasser l'enregistrement Cloudflare en

Proxied: l'avertissement d'exposition d'IP disparaît alors de lui-même.

Deux points en suspens

— Le nettoyage Git ne vaut que pour `production`.

Les `freddy`
branch
et `main`
suive `freddy`
toujo
les
207
fichier
d'out
Un
futur
merg
de
les
ramènerait.

— Le clone obsolète sur le serveur principal

reste `/var/www/DUGHU-WEB-3.0`
dans
, `main`
sur
,
sans
dépendances
installées.
Rien
ne
l'exécute.

Rapport établi le 21 août 2026 · branche production · HEAD 74eda2d · 6 commits poussés sur origin